

Théorie : Le langage C et le C pour Arduino

Le langage **C** est l'un des langages de programmation les plus utilisés pour la programmation embarquée, notamment avec **Arduino**. Il offre un **contrôle précis sur le matériel**, une **gestion optimisée des ressources** et une **exécution rapide**, contrairement aux langages interprétés comme **Python** que vous avez vu au Q1.

1. Le C : Langage « Bas-Niveau » mais Puissant

Le **C** est un **langage compilé**, ce qui signifie que le code source est transformé en langage machine avant d'être exécuté. Contrairement à **Python** qui est interprété, le C est plus rapide et permet de mieux gérer la **mémoire et les ressources**.

1.1 Pourquoi utiliser le C ?

- **Rapide et efficace** : Directement exécuté par le processeur après compilation.
- **Utilisation faible en ressources** : Idéal pour l'embarqué et les microcontrôleurs.
- **Portabilité** : Peut être utilisé sur différents matériels (PC, microcontrôleurs, Arduino, etc.).
- **Langage proche du matériel** : Permet un **contrôle direct des registres et des ports d'entrées/sorties**.
- Syntaxe utilisée dans **de nombreux langages dérivés (C++, Java, Arduino C, etc.)**.

1.2 Différences clés entre Python et C

Concept	Python	C Standard
Typage	Dynamique	Statique (int, float, char)
Compilation	Interprété	Compilé (gcc, clang)
Syntaxe	Indentation obligatoire	{ } pour structurer le code
Affichage	print("Hello")	printf("Hello\n");
Variables	Création dynamique	Déclaration avec type obligatoire
Boucles	for, while	Même logique, mais syntaxe différente et do while en bonus

2. Structure d'un Programme en C

Un programme en C suit une **structure bien définie** :

```
#include <stdio.h> // Inclusion des bibliothèques

// Fonction principale
int main() {
    printf("Hello, world!\n"); // Affichage à l'écran
    return 0; // Fin du programme
}
```

Explication :

1. `#include <stdio.h>` : Inclusion d'une bibliothèque standard (ici pour afficher du texte).
2. `int main()` : Point d'entrée du programme.
3. `{ ... }` : Délimite le bloc d'instructions.
4. `printf("Hello, world!\n");` : Affiche un message.
5. `return 0;` : Indique que le programme s'est terminé sans erreur. Ce `return` n'est pas obligatoire dans ce qu'on appelle une procédure mais bien dans une fonction.

2.1 Écrire un Programme en C Standard

- **Python**

```
print("Hello, world!")
```

- **C standard**

```
#include <stdio.h>
int main() {
    printf("Hello, world!\n");
    return 0;}
```

3. Variables et Types de Données en C

Une variable est une **zone mémoire** où l'on peut stocker une valeur.

3.1 Déclaration des variables

```
int entier = 10;
float decimal = 3.14;
char lettre = 'A';
```

Type	Taille	Exemple
int	2 ou 4 octets	int a = 5;
float	4 octets	float b = 3.14;
char	1 octet	char c = 'A';
bool	1 octet (Arduino)	bool etat = true;

⚠ Différence avec Python :

- En C, **on doit spécifier le type** (int, float, char).
- En Python, l'interpréteur **devine automatiquement** (x = 5).

4. Opérations et Conditions

4.1 Opérations mathématiques

```
int a = 5;
int b = 2;
int somme = a + b; // Addition
int produit = a * b; // Multiplication
int modulo = a % b; // Reste de la division
```

4.2 Entrées/Sorties et Variables

🔗 **Objectif** : Apprendre à manipuler les variables et les entrées utilisateur.

```
#include <stdio.h>
int main() {
    int nombre;
    printf("Entrez un nombre : ");
    scanf("%d", &nombre);
    printf("Vous avez entré : %d\n", nombre);
    return 0;}
```

- `scanf("%d", &nombre);` : & est obligatoire pour stocker la valeur.

4.3 Conditions (if, else, switch)

```
if (a > b) {
    printf("a est plus grand que b\n");
} else if (a == b) {
    printf("a et b sont égaux\n");
} else {
    printf("b est plus grand que a\n");
}
```

⚠ Différence avec Python :

- En Python : if a > b:
- En C : **Les conditions sont entourées de () et les blocs {} sont obligatoires.**

5. Boucles (for, while, do-while)

Les boucles permettent **d'exécuter plusieurs fois** un bloc de code.

5.1 Boucle for

```
for (int i = 0; i < 5; i++) {  
    printf("i = %d\n", i);  
}
```

💡 **Explication :**

1. `int i = 0;` → Initialisation
2. `i < 5;` → Condition de sortie
3. `i++` → Incrémentation

5.2 Boucle while

```
int compteur = 0;  
while (compteur < 5) {  
    printf("Compteur : %d\n", compteur);  
    compteur++;  
}
```

💡 La boucle continue **tant que la condition est vraie**.

5.3 Boucle do-while

```
int compteur = 0;  
do {  
    printf("Compteur : %d\n", compteur);  
    compteur++;  
} while (compteur < 5);
```

💡 **Différence avec while :** La boucle do-while **s'exécute au moins une fois** même si la condition est fausse.

6. Fonctions en C

📌 **Objectif** : Réutiliser du code pour éviter la répétition. C'est comme en Python au Q1, cela aide la modularité de votre code et sera SUPER utile. En Arduino vous en aurez quelques-unes de bases.

Pour revenir sur la théorie pure, une fonction est un ensemble de code qui va vous permettre un retour (le return en C). Ce retour devra être du type de la fonction. Dans un exemple, une fonction pourrait être de type int, le retour doit donc être un entier.

Le retour de la fonction fonctionne comme une variable de ce même type. Vous pouvez bien entendu aussi avoir des types comme float ou char.

Une fonction qui serait de type void (vide), serait en faite une procédure. Un bout de code qui va faire des choses mais ne donne pas de retour.

6.1 Fonction simple (ici on parlera de procédure car il n'y a pas de type)

```
void direBonjour() {  
    printf("Bonjour !\n");  
}  
  
int main() {  
    direBonjour();  
    return 0;  
}
```

6.2 Fonction avec paramètres

Il est possible de typer une fonction (lui donner un type de retour ici int) et de fournir des paramètres à nos fonctions pour travailler sur des données spécifiques au cours de cette fonction.

```
int addition(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int resultat = addition(5, 3);  
    printf("Résultat : %d\n", resultat);  
    return 0;  
}
```

7. Le C sur Arduino

📌 **Objectif** : Adapter le C pour un microcontrôleur.

7.1 Différences avec le C standard

Concept	C Standard	C pour Arduino
Fonction principale	main()	setup() + loop()
Affichage	printf()	Serial.print()
Temps	Horloge système	delay(ms)
Entrées/Sorties	scanf(), printf()	digitalRead(), digitalWrite()

7.2 La Structure d'un Programme Arduino

```
void setup() {  
    Serial.begin(9600); // Initialisation du moniteur série  
}  
void loop() {  
    Serial.println("Boucle en cours...");  
    delay(1000); // Attendre 1 seconde  
}
```

Explication :

- setup(): S'exécute une seule fois au démarrage.
- loop(): Tourne en boucle infinie.

7.3 Exemple : LED clignotante

```
int led = 13;  
  
void setup() {  
    pinMode(led, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(led, HIGH);  
    delay(1000);  
    digitalWrite(led, LOW);  
    delay(1000);  
}
```

⚡ **Explication :**

- setup() est exécuté une seule fois.
- loop() tourne en boucle.
- pinMode(led, OUTPUT); définit la broche 13 comme une sortie.
- ⚡ **Exercice** : Ajouter un bouton sur la broche 2 et allumer la LED uniquement si le bouton est pressé.

```

int bouton = 2;
int led = 13;
void setup() {
    pinMode(bouton, INPUT_PULLUP); // Activer la résistance interne de pull-up
    pinMode(led, OUTPUT);
}
void loop() {
    if (digitalRead(bouton) == LOW) { // Si le bouton est pressé
        digitalWrite(led, HIGH);
    } else {
        digitalWrite(led, LOW);
    }
}

```

7.4 Structurer le Code avec des Fonctions en C Arduino

🔗 **Objectif** : Organiser le code avec des fonctions réutilisables, apprentissage de la modularité.

7.4.1 Fonction Simple

```

void allumerLED() {
    digitalWrite(13, HIGH);
}
void setup() {
    pinMode(13, OUTPUT);
}
void loop() {
    allumerLED();
    delay(1000);
    digitalWrite(13, LOW);
    delay(1000);
}

```

7.4.2 Fonction avec Paramètres

```

void clignoterLED(int pin, int delai) {
    digitalWrite(pin, HIGH);
    delay(delai);
    digitalWrite(pin, LOW);
    delay(delai);
}
void setup() {
    pinMode(13, OUTPUT);
}
void loop() {
    clignoterLED(13, 500);
}

```

8. Gestion Mémoire et Pointeurs

Pourquoi les pointeurs sont-ils importants ?

- Permettent de **manipuler directement la mémoire**.
- Utiles pour **optimiser la RAM limitée d'un Arduino**.

8.1 Déclaration de pointeurs

```
int x = 10;
int *ptr = &x;

void setup() {
    Serial.begin(9600);
    Serial.print("Valeur de x : ");
    Serial.println(x);
    Serial.print("Adresse de x : ");
    Serial.println((int)ptr);
}
```

8.2 Utilisation des Pointeurs avec des Fonctions

```
void doublerValeur(int *nombre) {
    *nombre = *nombre * 2;
}

void setup() {
    Serial.begin(9600);

    int valeur = 5;

    doublerValeur(&valeur);

    Serial.print("Nouvelle valeur : ");

    Serial.println(valeur);
}
```